

## Code-Layout, Readability and Reusability

|                      |                                                     |
|----------------------|-----------------------------------------------------|
| <b>Date</b>          | 13 July 2026                                        |
| <b>Team ID</b>       | XXXXXX                                              |
| <b>Project Name</b>  | EduGenie – Google Gemini Powered Learning Assistant |
| <b>Maximum Marks</b> | 5 Marks                                             |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                     |
|------|---------------------------|-----------------------------------------------|-------------------------------|-----------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                           | Proper Python indentation maintained throughout the project.                |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Code organized into modules, templates, static, and main application files. |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Clear variable names such as query, query_type, response.                   |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions are descriptive (e.g., answer_question(), generate_quiz()).       |
| 5    | Code Comments             | Inline and block comments explain logic       | Partial                       | Important sections are documented; additional comments can be added.        |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Features are separated into independent modules (QA, Quiz, Summary, etc.).  |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | Reusable functions reduce duplicate code.                                   |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                           | API exceptions and invalid inputs are handled gracefully.                   |

Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused             | Reusability Level (High / Medium / Low) |
|------|-------------------------|-----------------------|--------------------------|-----------------------------------------|
| 1    | Q&A Module              | Python, Gemini API    | Answer Generation        | High                                    |
| 2    | Explanation Module      | Python, Hugging Face  | Topic Explanation        | High                                    |
| 3    | Quiz Module             | Python                | Quiz Generation          | High                                    |
| 4    | Summary Module          | Python                | Notes Summarization      | High                                    |
| 5    | Recommendation Module   | Python                | Learning Recommendations | High                                    |

Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                    |
|--------------------------|--------------|-------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-organized folder structure and modular implementation. |
| Readability              | 5            | Easy-to-understand naming conventions and clean code.       |
| Reusability              | 5            | Independent modules can be reused across features.          |
| Documentation / Comments | 4            | Basic documentation included; can be further improved.      |
| Overall Score            | 4.8/5        | Clean, modular, maintainable, and scalable implementation.  |